Neo4J



- Neo4j es un software libre de Base de datos orientada a grafos, implementado en Java
- Los desarrolladores describen a Neo4j como un motor de persistencia embebido, basado en disco, implementado en Java, completamente transaccional, que almacena datos estructurados en grafos en lugar de en tablas
- La versión 1.0 de Neo4j fue lanzada en febrero de 2010. Neo4j fue desarrollado por Neo Technology, una startup sueca con base en Malmö y San Francisco Bay Area en Estados Unidos

Neo4J

- Quien hable de Big Data, seguramente dirá que lo más importante es extraer valor de la información (es el concepto perfecto en el que se engloban las cuatro famosas V de los grandes datos: volumen, velocidad, variedad y veracidad)
- Muchas empresas tienen datos que sirven de poco porque no están estructurados, no conocen la relación entre ellos
- Las bases de datos orientadas a grafos (BDOG) ayudan a encontrar relaciones

Cómo funciona Neo4j y cuáles son sus ventajas

- Neo4j usa grafos para representar datos y las relaciones entre ellos. Un grafo se define como cualquier representación gráfica formada por vértices y aristas. Dentro de estas representaciones gráficas, tenemos varios tipos de grafos:
- **Grafos no dirigidos**: los nodos y las relaciones son intercambiables, su relación se puede interpretar en cualquier sentido. Las relaciones de amistad en la red social Facebook, por ejemplo, son de este tipo
- **Grafos dirigidos**: los nodos y la relaciones no son bidireccionales por defecto. Las relaciones en Twitter son de este tipo. Un usuario puede seguir a determinados perfiles en esta red social sin que ellos le sigan a él

Cómo funciona Neo4j y cuáles son sus ventajas

- **Grafos con peso**: en este tipo de grafos las relaciones entre nodos tienen algún tipo de valoración numérica. Eso permite luego hacer operaciones
- **Grafos con etiquetas**: estos grafos llevan incorporadas etiquetas que pueden definir los distintos vértices y también las relaciones entre ellos. En Facebook podríamos tener nodos definidos por términos como 'amigo' o 'compañero de trabajo' y la relaciones como 'amigo de' o 'socio de'
- **Grafos de propiedad**: es un grafo con peso, con etiquetas y donde podemos asignar propiedades tanto a nodos como relaciones (por ejemplo, cuestiones como nombre, edad, país de residencia, nacimiento). Es el más complejo

Cómo funciona Neo4j y cuáles son sus ventajas

- Neo4j utiliza grafos de propiedad para extraer valor añadido de los datos de cualquier empresa con
  - gran rendimiento
  - de una forma ágil
  - flexible y escalable

Rendimiento

- Las bases de datos orientadas a grafos como Neo4j tienen mejor rendimiento que las relacionales (SQL) y las no relacionales (NoSQL). La clave es que, aunque las consultas de datos aumenten exponencialmente, el rendimiento de Neo4j no desciende, frente a lo que sí sucede con las BD relacionales como MySQL
- Las BDOG responden a las consultas actualizando el nodo y la relaciones de esa búsqueda y no todo el grafo completo. Eso optimiza mucho el proceso

Agilidad

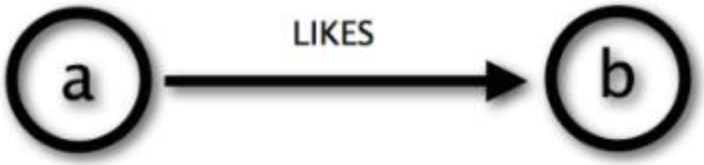
- Neo4J tiene muchas ventajas, pero una es su agilidad en la gestión de datos
- Si nosotros quisiéramos llevar al límite sus capacidades, tendríamos que superar un volumen total de 34.000 millones de nodos (datos), 34.000 millones de relaciones entre esos datos, 68.000 millones de propiedades y 32.000 tipos de relaciones

Flexibilidad y escalabilidad

- Cuando los desarrolladores de una empresa trabajan con grandes datos, buscan flexibilidad y escalabilidad
- Las bases de datos orientadas a grafos aportan mucho en este sentido porque cuando aumentan las necesidades, las posibilidades de añadir más nodos y relaciones a un grafo ya existente son enormes

Cypher: Lenguaje de Consulta

Cypher using relationship 'likes'



Cypher

```
(a) -[:LIKES]-> (b)
```

Cypher: Modelado de Nodos

```
MATCH (node:Label) RETURN node.property

MATCH (node1:Label1)-->(node2:Label2)
WHERE node1.propertyA = {value}
RETURN node2.propertyA, node2.propertyB
```

Cypher: Modelado de Aristas

Relaciones

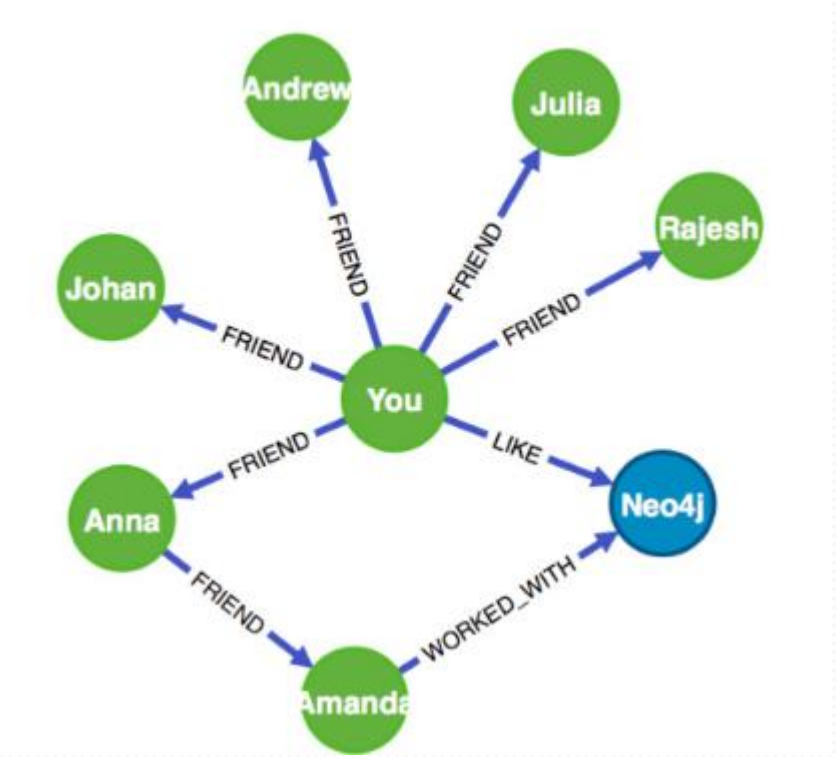
```
MATCH (n1:Label1)-[rel:TYPE]->(n2:Label2)
WHERE rel.property > {value}
RETURN rel.property, type(rel)
```

Create Your Friends

```
MATCH (you:Person {name:"You"})
FOREACH (name in ["Johan","Rajesh","Anna","Julia","Andrew"] |
  CREATE (you)-[:FRIEND]->(:Person {name:name}))
```

Cypher: Ejemplo

- Encuentre a alguien en su red que pueda ayudarlo a aprender Neo4j



Cypher: Ejemplo

```
MATCH (you {name:"You"})
MATCH (expert)-[:WORKED_WITH]->(db:Database {name:"Neo4j"})
MATCH path = shortestPath( (you)-[:FRIEND*..5]-(expert) )
RETURN db,expert,path
```

Quiénes usan Neo4J

- Walmart
- Telenor
- UBS
- Cisco
- Hewlett-Packard
- Lufthansa
- Entre otros…



Neo4j: Graphs for Everyone

Neo4j es un sistema de gestión de bases de datos de grafos desarrollado por **Neo4j Inc**. Sus desarrolladores la describen como una base de datos transaccional compatible con ACID con almacenamiento y procesamiento de grafos nativos, Neo4j es la base de datos de grafos más popular según la clasificación del DB-Engines.

https://db-engines.com/en/system/Neo4j

Rank #21 Overall (#19 2022-04)
#1 Graph DBMS

License: Source code: GPLv3 and AGPLv3; Binaries: Freemium registerware
Developer(s): Neo Technology
Stable release: 5.6, March 2023
Written in: **Java** 73.0%, **Scala** 26.5%, ..
Initial release: **2007** (initial development began in 2003, but it has been publicly available since 2007)

https://github.com/neo4j/neo4j Stars: 11.3 K

https://www.linkedin.com/company/neo4j/ **784** empleados en LinkedIn, 45.482 seguidores

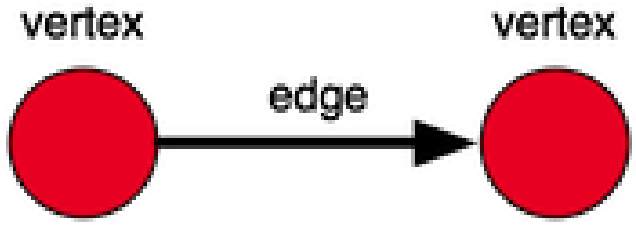
Neo4j highlights

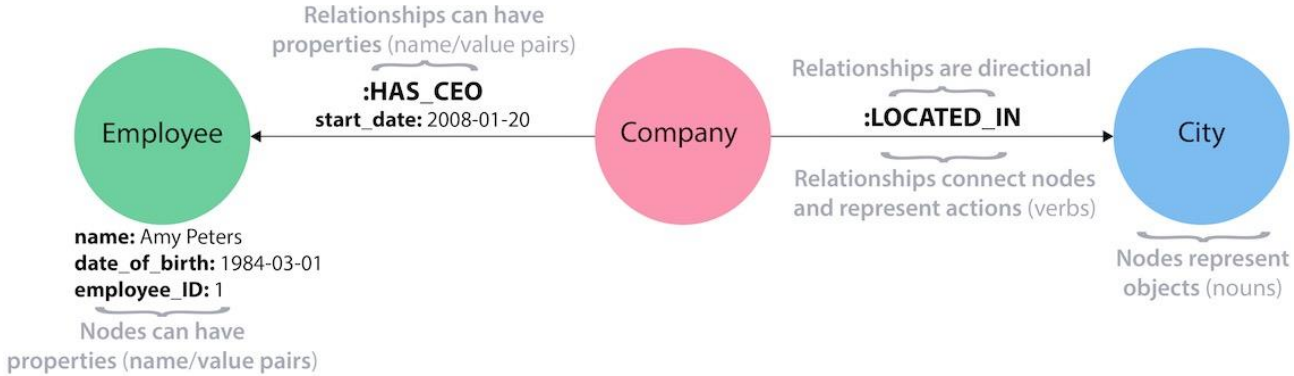
1. A native graph database
2. Whiteboard friendly
3. Supports rapid development
4. Provides true data safety through ACID transactions
5. Designed for business-critical and high-performance operations

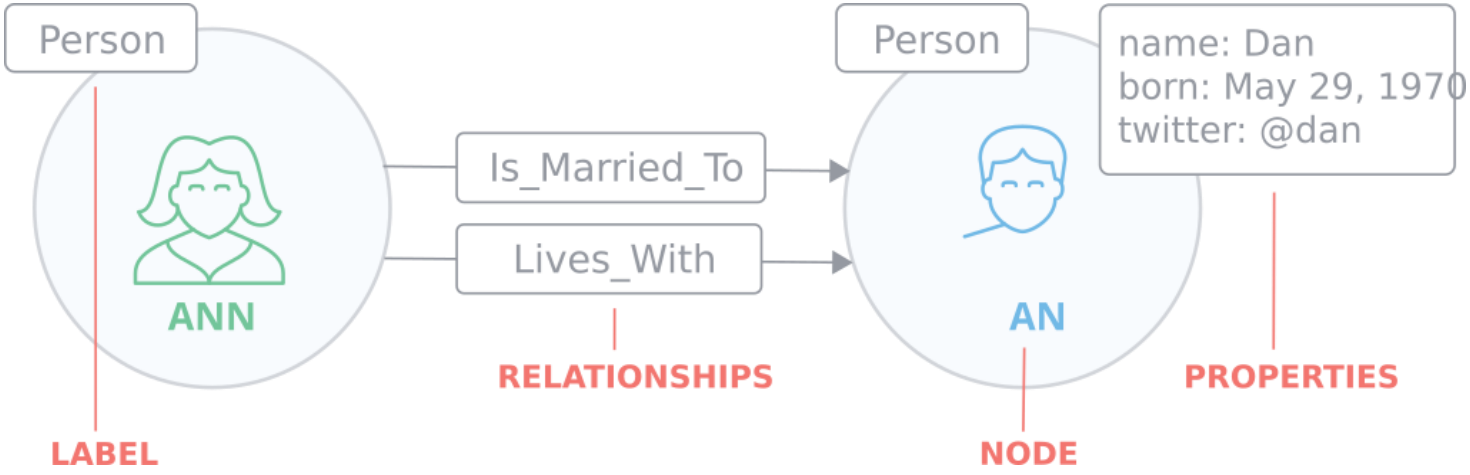
Graph Database

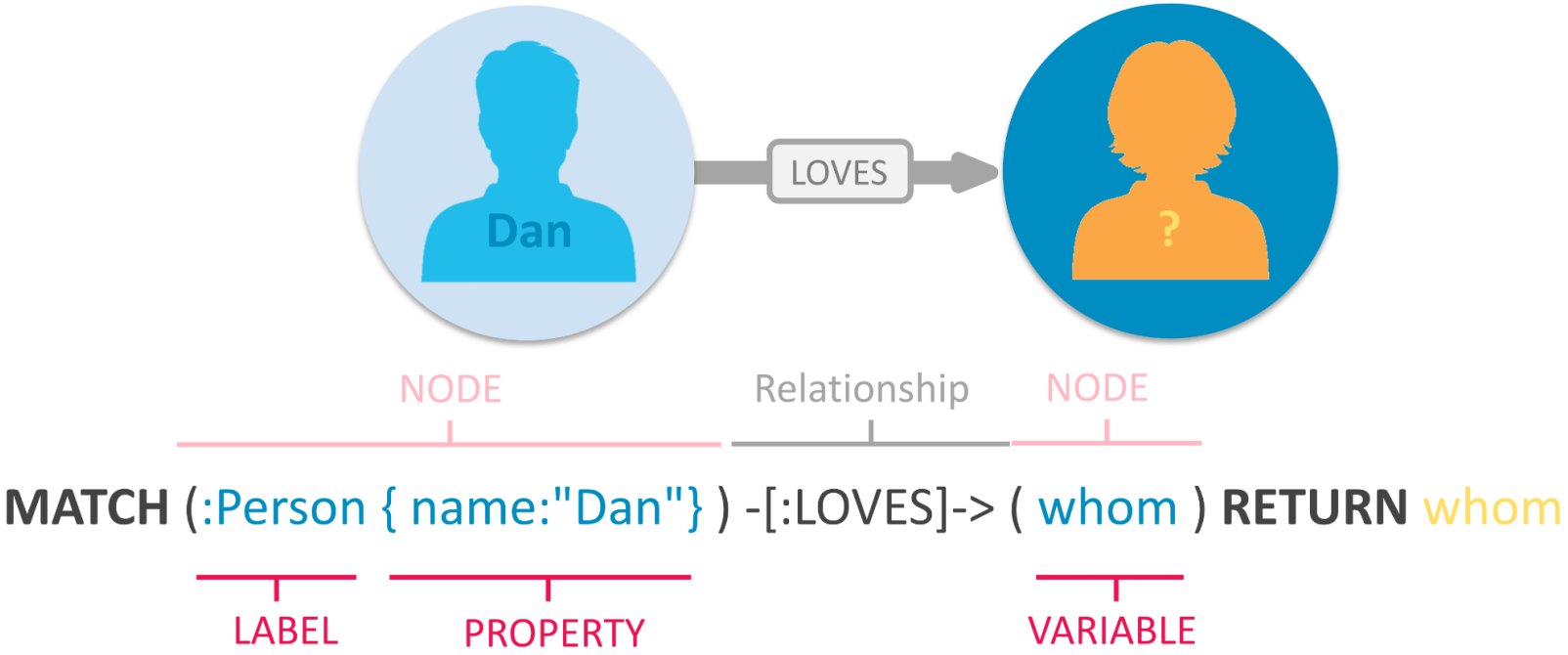
Neo4j is a member of a family of databases known as *graph databases* because it stores data as a graph (in the mathematical sense).

Una *graph database es* una base de datos que usa estructuras basadas en grafos (como por ejemplo **nodos**, **relaciones** y **propiedades**) para representar y almacenar los datos.




Relationships can have properties (name/value pairs)
:HAS_CEO
start_date: 2008-01-20
Employee
Company
Relationships are directional
:LOCATED_IN
City
Relationships connect nodes and represent actions (verbs)
Nodes represent objects (nouns)
name: Amy Peters
date_of_birth: 1984-03-01
employee_ID: 1
Nodes can have properties (name/value pairs)


Person
ANN
Is_Married_To
Lives_With
RELATIONSHIPS
LABEL
Person
AN
NODE
name: Dan
born: May 29, 1970
twitter: @dan
PROPERTIES



Graph database concepts

1. Nodos
2. Relaciones
3. Propiedades (son los atributos de los nodos y las relaciones), es una (key, value), donde key=name. (ver los diferentes tipos de datos)
4. Labels (los labels asignan roles o tipos a los nodos y a las relaciones)

Example Data Models → https://neo4j.com/graphgists/

small & big
Neo4j is small enough to be embedded into nearly any application; on the other side of the spectrum, Neo4j can run in large clusters of servers using master-slave replication and store tens of billions of nodes and as many relationships. In other words, Neo4j can handle just about any size problem that you can throw at it.

Server operating systems
[Linux, OS X, Solaris, Windows]

Cypher
Para interactuar con Neo4j usamos un lenguaje de consulta llamado ***Cypher*** (no es el único modo, existen otros lenguajes de consulta, como Gremlin)

https://en.wikipedia.org/wiki/Gremlin_(query_language)

GPL Community Edition OR *Enterprise Edition*.
Para el día 1 y 2 podemos usar la versión *GPL Community edition*, solo el día 3: que se va a ver algo de *Distributed High Availability*, necesitamos la versión EE.

https://jdk.java.net/archive/

EE (1er solapa) y **Community (2da solapa)** → https://neo4j.com/download-center/#releases

Neo4j Community Edition 4.x

1. Download
2. Extract (tar -xf neo4j-community-4.*<version>*-unix.tar.gz)
3. To run Neo4j in a background process, use:
    a. *<NEO4J_HOME>*/bin/neo4j start
4. To run Neo4j as a console application, use:
    a. <NEO4J_HOME>/bin/neo4j console
5. Visit http://localhost:7474 in your web browser.
6. Connect using the username **neo4j** with default password **neo4j**. You'll then be prompted to change the password.

Para la versión 4.x de Neo4J, hay que tener Java 11 (si tienen la última versión de Java, no funciona)

Guía para el uso de *Cypher*.

Una consulta en Cypher, luce así:

```
$ MATCH [some set of nodes and/or relationships]
WHERE [some set of properties holds]
RETURN [some set of results captured by the MATCH and WHERE clauses]
```

```
// creando 3 nodos (2 vinos y 1 publicación)
$ CREATE (w:Wine {name:"Prancing Wolf", style: "ice wine", vintage: 2015})
$ CREATE (w:Wine {name:"Prancing Wolf", style: "red wine", vintage: 2016})
$ CREATE (p:Publication {name: "Wine Expert Monthly"})

// creando 1 relación, entre los 2 nodos.
MATCH (p:Publication {name: "Wine Expert Monthly"}), (w:Wine {name: "Prancing Wolf"})
CREATE (p)-[r:reported_on]->(w)

// seleccionando todos los registros del grafo
$ MATCH (n) RETURN n;
```

http://localhost:7474/db/data/relationship/**0**	$ MATCH ()-[r]-() WHERE id(r) = **0** RETURN r

http://localhost:7474/db/data/node/0	$ MATCH (a)-[r]-(b) WHERE id(a) = 0 RETURN a

```
$ MATCH (a)-[]-() WHERE id(a) = 0 RETURN a

// voy a borrar la relación que habíamos creado entre los 2 nodos.
$ MATCH ()-[r:reported_on]->() DELETE r

$ MATCH (n) RETURN n

// y la vuelvo a crear con un rating=97
$ MATCH (p:Publication {name: "Wine Expert Monthly"}),(w:Wine {name: "Prancing Wolf"})
CREATE (p)-[r:reported_on {rating: 97}]->(w)

$ CREATE (g:GrapeType {name: "Riesling"})

$ MATCH (w:Wine {name: "Prancing Wolf"}),(g:GrapeType {name: "Riesling"})
CREATE (w)-[r:grape_type]->(g)
```

DELETE

```
// creo un Nodo y una relación (efímera)
//
$ CREATE (e:EphemeralNode {name: "short lived"})
$ MATCH (w:Wine {name: "Prancing Wolf"}), (e:EphemeralNode {name: "short lived"}) CREATE (w)-
[r:short_lived_relationship]->(e)

$ MATCH ()-[r:short_lived_relationship]-() DELETE r
$ MATCH (e:EphemeralNode) DELETE e
```

Si queremos empezar de cero, borrando todos los nodos y los ejes, deberíamos ejecutar esto:

```
$ MATCH (n) OPTIONAL MATCH (n)-[r]-() DELETE n, r
o
$ MATCH (n) DETACH DELETE n;
```

Si queremos borrar todo y el grafo es muy grande, les recomiendo leer este artículo: [Large Delete Transaction Best Practices in Neo4j]

Nota: Si queremos borrar **todo**, tienen que borrar el directorio donde está la base de datos, en mi caso es este directorio: C:\Users*<username>*\Documents\Neo4j\default.graphdb

Vamos a armar otra vez el grafo, con 3 vinos, una bodega/vineria, un tipo de uva

```
// creo un vino
CREATE (w:Wine {name:"Prancing Wolf", style: "ice wine", vintage: 2015})

// creo una revista
CREATE (p:Publication {name: "Wine Expert Monthly"})

// creo la relación con un rating de 97 puntos
MATCH (p:Publication {name: "Wine Expert Monthly"}), (w:Wine {name: "Prancing Wolf"}) CREATE (p)-[r:reported_on
{rating: 97}]->(w)

// creo un tipo de uva
CREATE (g:GrapeType {name: "Riesling"})

// creo la bodega que produce el vino Prancing Wolf
CREATE (wr:Winery {name: "Prancing Wolf Winery"})
```

```
// creo 2 vinos
CREATE (w:Wine {name:"Prancing Wolf", style: "Kabinett", vintage: 2002})
CREATE (w:Wine {name: "Prancing Wolf", style: "Spätlese", vintage: 2010})

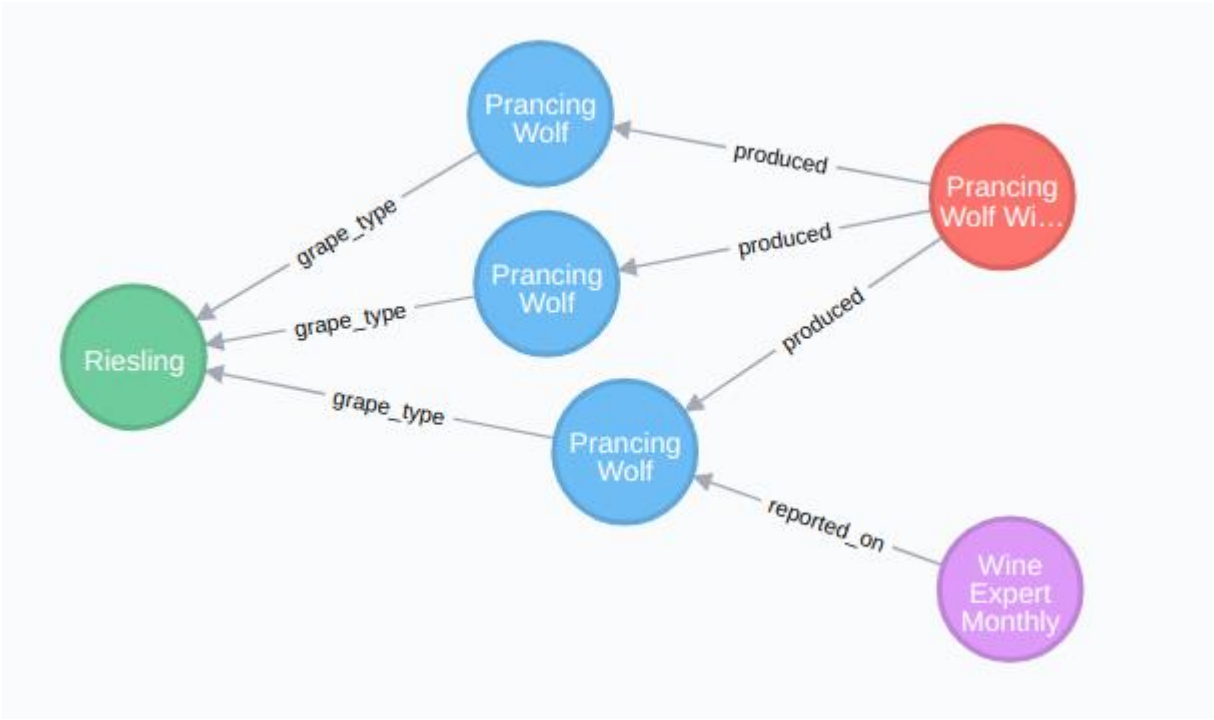
// creo la relación entre vino y la bodega
$ MATCH (wr:Winery {name: "Prancing Wolf Winery"}),(w:Wine {name: "Prancing Wolf"})
  CREATE (wr)-[r:produced]->(w)

$ MATCH (w:Wine),(g:GrapeType {name: "Riesling"})
  CREATE (w)-[r:grape_type]->(g)
```

Le subo el puntaje al vino (**UPDATE**)

```
$ MATCH (p:Publication)-[b:reported_on]-(w:Wine)
SET b.rating = 98
RETURN p,b,w
```

Nos debería quedar algo asi

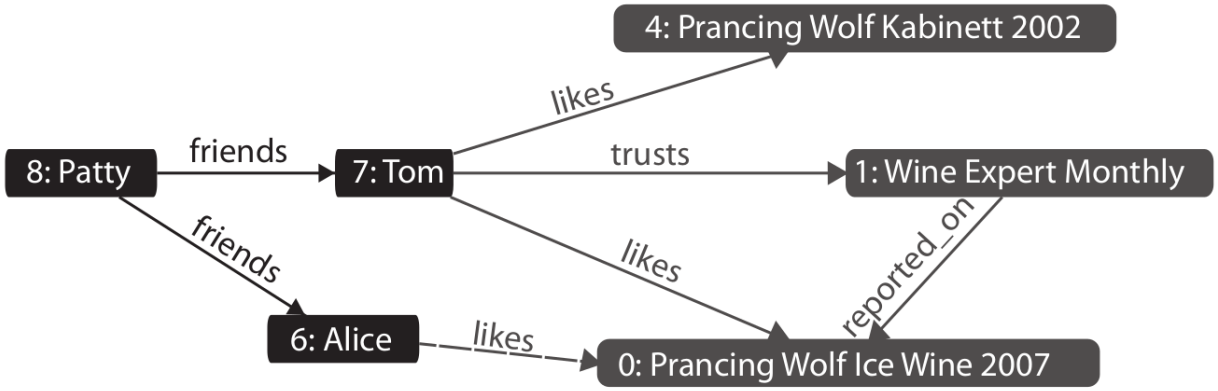


Si queremos listar los vinos que fueron publicados en la revista "Wine Expert Monthly"

La query sería el siguiente:

```
$ MATCH (p:Publication)-[r:reported_on]-(v:Wine)
RETURN v
```

Vamos a agregar 3 personas (Tom, Alice y Patty) y sus relaciones.




```
$ CREATE (p:Person {name: "Alice"})
$ MATCH (p:Person {name: "Alice"}), (w:Wine {name: "Prancing Wolf", style: "ice wine"})
  CREATE (p)-[r:likes]->(w)

$ CREATE (p:Person {name: "Tom"})
$ MATCH (p:Person {name: "Tom"}), (w:Wine {name: "Prancing Wolf", style: "ice wine"})
  CREATE (p)-[r:likes]->(w)
$ MATCH (p:Person {name: "Tom"}), (w:Wine {name: "Prancing Wolf", style: "Kabinett"})
  CREATE (p)-[r:likes]->(w)
$ MATCH (p:Person {name: "Tom"}),(pub:Publication {name: "Wine Expert Monthly"})
  CREATE (p)-[r:trusts]->(pub)

$ CREATE (p:Person {name: "Patty"})
$ MATCH (p1:Person {name: "Patty"}), (p2:Person {name: "Tom"}) CREATE (p1)-[r:friends]->(p2)
$ MATCH (p1:Person {name: "Patty"}), (p2:Person {name: "Alice"}) CREATE (p1)-[r:friends]->(p2)

Probar el operador --> con Alice y con Patty
$ MATCH (p:Person {name: "Alice"})-->(n) RETURN n;                    // acá encontramos un vino
$ MATCH (p1:Person {name: "Alice"})-->(p2:Person) RETURN p2.name;     // aca no encontramos nada
```

```
$ MATCH (p1:Person {name: "Patty"})-->(p2:Person) RETURN p2.name;      // encontramos a Alice y a Tom

$ MATCH (p:Person) WHERE p.name <> 'Patty' RETURN p;
```

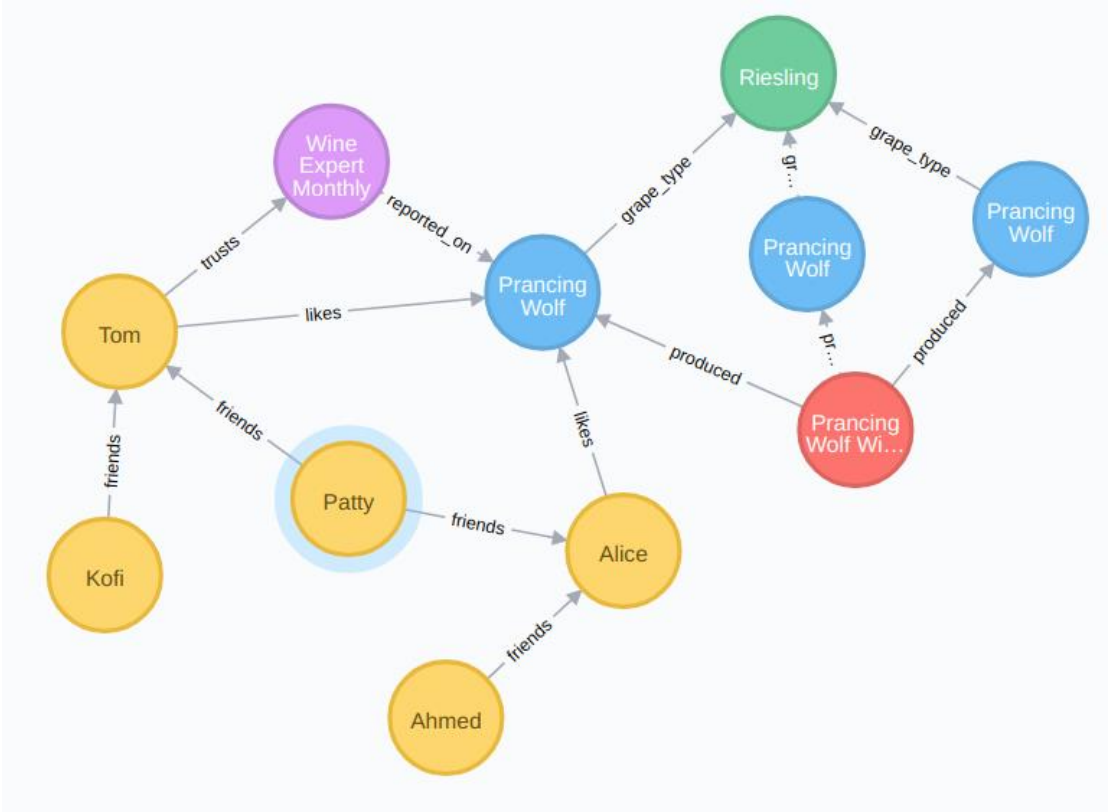
Vamos a crear más nodos y relaciones

```
// Ahmed y Kofi son personas
$ CREATE (p1:Person {name: "Ahmed"}), (p2:Person {name: "Kofi"});

// Ahmed es amigo de Alice
$ MATCH (p1:Person {name: "Ahmed"}),(p2:Person {name: "Alice"})
CREATE (p1)-[r:friends]->(p2);

// Tom es amigo de Kofi
$ MATCH (p1:Person {name: "Kofi"}),(p2:Person {name: "Tom"})
CREATE (p1)-[r:friends]->(p2);
```

Buscando a los amigos de los amigos de Patty (fof = friend of friend)




```
$ MATCH (fof:Person) - [f1:friends] - (p1:Person) - [f2:friends] - (p:Person {name: "Patty"})
RETURN fof.name;

// en este caso particular, es igual a esta otra consulta (aca le estoy poniendo una dirección a la relación de amistad)
$ MATCH (fof:Person) - [:friends] -> (f:Person) <- [:friends] - (p:Person {name: "Patty"})
RETURN fof.name;
```

[Modeling Data in Neo4j: Bidirectional Relationships]

Índices/*Indexes*, Restricciones/*Constraints* y "esquemas/*Schemas*" en Cypher.

```
// para crear un índice sobre el atributo name de todos los nodos de tipo Wine.
$ CREATE INDEX FOR (w:Wine) ON (w.name);

$ :schema  // para ver los indices y los contraints

// para eliminar el índice
$ DROP INDEX index_name [IF EXISTS]
```

Restricciones/*Constraints*

Si creo esta restricción

Person

```
$ CREATE CONSTRAINT FOR (p:Publication) REQUIRE p.name IS UNIQUE;

$ CREATE (p:Person {name: "Tom"});
Neo.ClientError.Schema.ConstraintValidationFailed: Node(40) already exists with label `Person` and property `name`
= 'Tom'
```

No voy a poder tener dos nodos, con el mismo nombre.

Si queremos borrar la restricción.

```
$ :schema   // miramos el nombre de la restriccion que queremos borrar.
$ DROP CONSTRAINT constraint_df66ae8
```

Schemas

*Neo4j doesn't enable you to enforce hard schemas the way that relational databases do, but it does enable you to provide some structure to nodes in your graphs by creating **indexes** and **constraints** for specified **labels**.*

Para acceder al esquema, tenemos que ejecutar la siguiente sentencia.

$:schema

```
$ :schema
```

```
Indexes
   ON :Person(name) ONLINE  (for uniqueness constraint)

Constraints
   ON ( person:Person ) ASSERT person.name IS UNIQUE
```

more info: https://neo4j.com/docs/developer-manual/current/cypher/schema/

Functions in Cypher → http://neo4j.com/docs/developer-manual/current/cypher/functions/

(por ej.)
Spatial Queries - Calculando la distancia entre dos puntos.

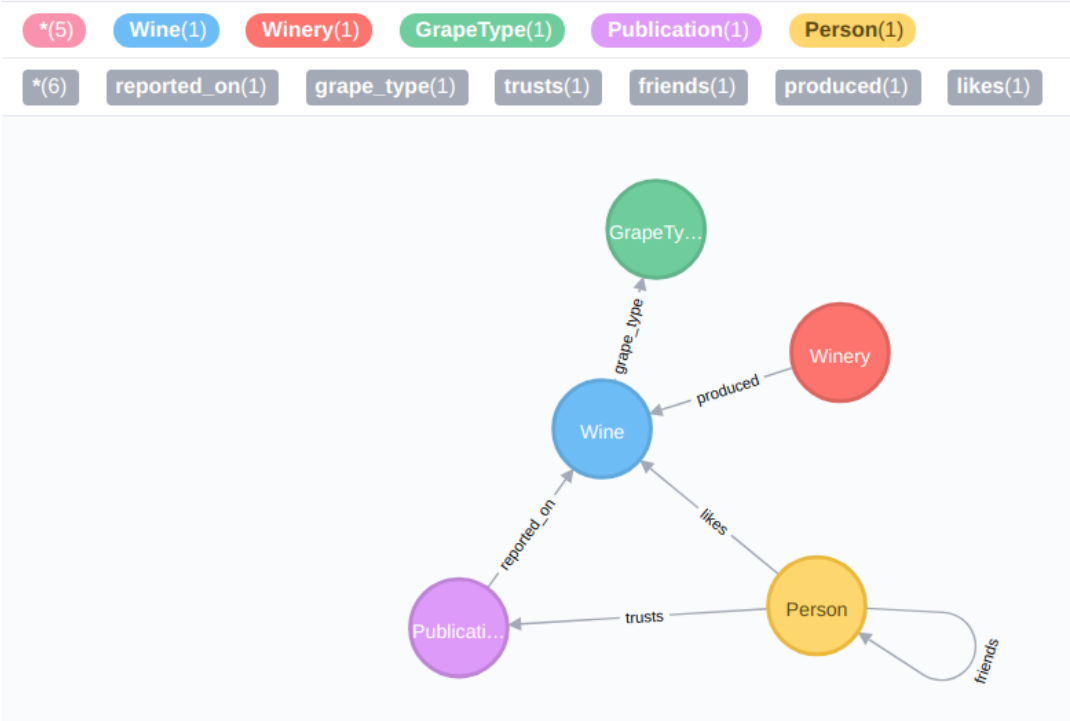
```
RETURN distance(
  point({ longitude: -58.479259, latitude: -34.582507 }),
  point({ longitude: -58.479198, latitude: -34.581724 })
)
```

87.34 (mts)

```
// Show meta-graph
```

```
CALL db.schema()        (v3.5, deprecada en la 4.x)

// para el Sandbox (nuevas versiones)
CALL db.schema.visualization()
```



Count all nodes	MATCH (n) RETURN count(n)
Count all relationships	MATCH ()-->()

	RETURN count(*);
List node labels	CALL db.labels()
List relationship types	CALL db.relationshipTypes()

REST, Indexes, Algorithms

REST (cURL)

Los siguientes comandos los vamos a ejecutar desde la Terminal

```
// ejemplo del libro
//
$ curl http://localhost:7474/db/data/
{
  "errors" : [ {
    "code" : "Neo.ClientError.Security.Unauthorized",
    "message" : "No authentication header supplied."
  } ]
}

Modificar el archivo /conf/neo4j.conf
# dbms.security.auth_enabled=false
```

o

```
$ curl -u neo4j http://localhost:7474/db/data/
Enter host password for user 'neo4j': ******
{
  "extensions" : { },
  "node" : "http://localhost:7474/db/data/node",
  "relationship" : "http://localhost:7474/db/data/relationship",
  "node_index" : "http://localhost:7474/db/data/index/node",
  "relationship_index" : "http://localhost:7474/db/data/index/relationship",
  "extensions_info" : "http://localhost:7474/db/data/ext",
  "relationship_types" : "http://localhost:7474/db/data/relationship/types",
  "batch" : "http://localhost:7474/db/data/batch",
  "cypher" : "http://localhost:7474/db/data/cypher",
  "indexes" : "http://localhost:7474/db/data/schema/index",
  "constraints" : "http://localhost:7474/db/data/schema/constraint",
  "transaction" : "http://localhost:7474/db/data/transaction",
  "node_labels" : "http://localhost:7474/db/data/labels",
  "neo4j_version" : "3.4.9"
}
```

Node Labels

```
$ curl http://localhost:7474/db/data/labels
[ "Winery", "Person", "Loc", "User", "Page", "Wine", "Publication", "GrapeType" ]
```

Relationship Types

```
$ curl http://localhost:7474/db/data/relationship/types
["reported_on","grape_type","produced","likes","trusts","friends"]
```

Creating Nodes and Relationships Using REST

```
$ curl -i -XPOST http://localhost:7474/db/data/node -H "Content-Type: application/json" -d '{ "name": "P.G.
Wodehouse", "genre": "British Humour" }'
```

Observar el resultado del curl, para determinar el "id" que se le asignó a este nodo.

```
HTTP/1.1 201 Created
Date: Sun, 21 Oct 2018 20:57:06 GMT
Access-Control-Allow-Origin: *
Location: http://localhost:7474/db/data/node/2
Content-Type: application/json;charset=utf-8
Content-Length: 1253

{
  "extensions" : { },
  "metadata" : {
    "id" : 2,
    "labels" : [ ]
  },
  "paged_traverse" : "http://localhost:7474/db/data/node/2/paged/traverse/{returnType}{?pageSize,leaseTime}",
  "outgoing_relationships" : "http://localhost:7474/db/data/node/2/relationships/out",
  "outgoing_typed_relationships" : "http://localhost:7474/db/data/node/2/relationships/out/{-list|&|types}",
  "create_relationship" : "http://localhost:7474/db/data/node/2/relationships",
  "labels" : "http://localhost:7474/db/data/node/2/labels",
  "traverse" : "http://localhost:7474/db/data/node/2/traverse/{returnType}",
  "all_relationships" : "http://localhost:7474/db/data/node/2/relationships/all",
  "all_typed_relationships" : "http://localhost:7474/db/data/node/2/relationships/all/{-list|&|types}",
  "property" : "http://localhost:7474/db/data/node/2/properties/{key}",
  "self" : "http://localhost:7474/db/data/node/2",
  "incoming_relationships" : "http://localhost:7474/db/data/node/2/relationships/in",
  "properties" : "http://localhost:7474/db/data/node/2/properties",
```

```
  "incoming_typed_relationships" : "http://localhost:7474/db/data/node/2/relationships/in/{-list|&|types}",
  "data" : {
    "name" : "P.G. Wodehouse",
    "genre" : "British Humour"
  }
}
```

Podemos ver este nodo, en la siguiente url → http://localhost:7474/db/data/node/2
Podemos consultar un atributo o una propiedad del nodo, el 'genre' agregando a la 'properties' a la url

```
$ curl http://localhost:7474/db/data/node/2/properties/genre
"British Humour"
```

Agregando otro nodo, con estas propiedades: {"name" : "Jeeves Takes Charge", "style" : "short story" }

```
$ curl -i -XPOST http://localhost:7474/db/data/node -H "Content-Type: application/json" -d '{ "name" : "Jeeves
Takes Charge", "style" : "short story" }'
…
 id → 93
 "self" : "http://localhost:7474/db/data/node/93",
…
```

Voy a generar una relación, entre dos nodos (el nro 2 → 93)

```
$ curl -i -XPOST http://localhost:7474/db/data/node/2/relationships \
-H "Content-Type: application/json" \
-d '{
"to": "http://localhost:7474/db/data/node/93",
"type": "WROTE",
"data": {"published": "November 28, 1916"}
```

```
}'
…
Se generó la relación, con id 56.
…
```

Algoritmos

```
$ curl -X POST http://localhost:7474/db/data/node/2/paths \
-H "Content-Type: application/json" \
-d '{
"to": "http://localhost:7474/db/data/node/93",
"relationships": {"type": "WROTE"},
"algorithm": "shortestPath",
"max_depth": 10
}'
```

Resultado:
Usando la relación 56 en un solo paso, llegamos del id 2 al 93.


```
[ {
  "relationships" : [ "http://localhost:7474/db/data/relationship/56" ],
  "nodes" : [ "http://localhost:7474/db/data/node/2", "http://localhost:7474/db/data/node/93" ],
  "directions" : [ "->" ],
  "length" : 1,
  "start" : "http://localhost:7474/db/data/node/2",
  "end" : "http://localhost:7474/db/data/node/93"
} ]
```

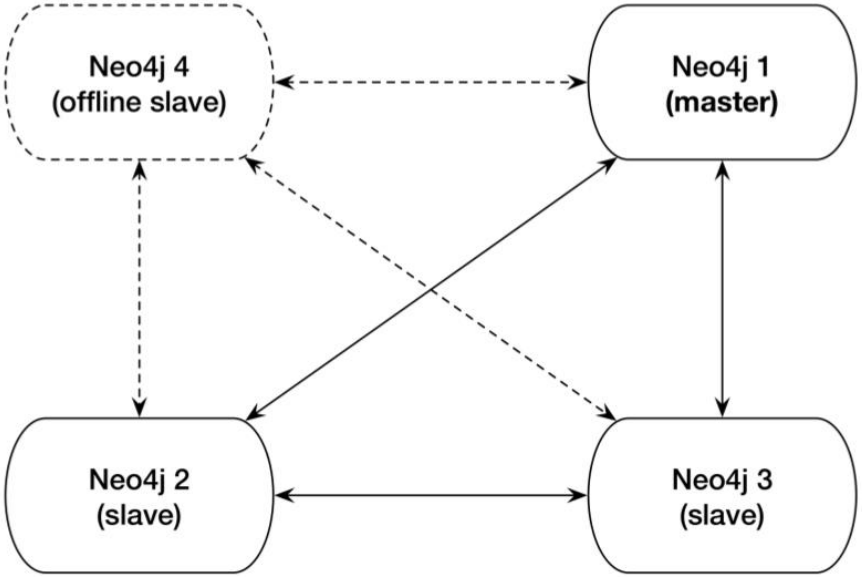

Probar el algoritmo ***shortestPath*** con otro par de nodos que no estén a distancia 1.

The other path algorithm choices are ***allPaths***, ***allSimplePaths***, and ***dijkstra***. You can find information on these algorithms in the online documentation, but detailed coverage is outside the scope of this book.

Distributed High Availability

Master y slaves
Slaves in Neo4J accept writes.
Slaves writes will synchronize with the master node.
Si un nodo master pasa a estar offline, algún nodo slave toma esa responsabilidad.
El nodo master, se elige de manera automática dentro del cluster.

*Neo4j clusters relied on **zookeeper** as an external coordination mechanism*



Bajarse el neo4j (ee), ubuntu → neo4j-enterprise-3.4.9-unix.tar.gz

```
$ tar fx neo4j-enterprise-3.4.9-unix.tar.gz
mv neo4j-enterprise-3.4.9 neo4j-1.local
cp -R neo4j-1.local neo4j-2.local
cp -R neo4j-1.local neo4j-3.local
```

Modificar el archivo de configuración en cada directorio.

```
dbms.mode=HA
dbms.memory.pagecache.size=200m
dbms.backup.address=127.0.0.1:6366
dbms.backup.enabled=true
ha.server_id=1
ha.initial_hosts=127.0.0.1:5001,127.0.0.1:5002,127.0.0.1:5003
ha.host.coordination=127.0.0.1:5001
ha.host.data=127.0.0.1:6363
dbms.connector.http.enabled=true
dbms.connector.http.listen_address=:7474
dbms.connector.bolt.enabled=true
dbms.connector.bolt.tls_level=OPTIONAL
dbms.connector.bolt.listen_address=:7687
dbms.security.auth_enabled=false
```

Copiar el mismo (conf) en el servidor 2 y 3, pero sumando 1 a los siguientes parámetros:

```
ha.server_id=1
ha.host.coordination=127.0.0.1:5001
ha.host.data=127.0.0.1:6363
```

```
dbms.connector.http.listen_address=:7474
dbms.connector.bolt.listen_address=:7687

Levantar los 3 nodos.

$ neo4j-1.local/bin/neo4j start
$ neo4j-2.local/bin/neo4j start
$ neo4j-3.local/bin/neo4j start

Ingresar al shell de uno de los 3 servidores (cualquiera de los 3)

$ neo4j-2.local/bin/cypher-shell

Insertar un registro
neo4j> CREATE (p:Person {name: "Homero"});
neo4j> :exit

Ingresar a otro servidor, por ej. el 3

$ neo4j-3.local/bin/cypher-shell

neo4j> CREATE (p:Person {name: "Bart"});
neo4j> match (n) return n;
match (n) return n;
+---------------------------+
| n                         |
+---------------------------+
| (:Person {name: "Bart"})  |
| (:Person {name: "Homero"}) |
+---------------------------+
```

```
neo4j> :exit
```

La información se replica entre los tres servidores, generando un ambiente de alta disponibilidad.

Backups

'bin/neo4j-admin backup'

```
$ neo4j-1.local/bin/neo4j-admin backup  \
  --from 127.0.0.1:6366                 \
  --name neo4j-`date +%Y.%m.%d`.db      \
  --backup-dir /mnt/backups
```

Se puede hacer un 'full backup' la primera vez y después un backup incremental.

Neo4j's Strengths

Neo4j is one of the finest examples of open source graph databases. Graph databases are perfect for unstructured data, in many ways even more so than document databases. Not only is Neo4j typeless and schemaless, but it puts no constraints on how data is related. It is, in the best sense, a free-for-all. Currently, Neo4j can support 34.4 billion nodes and 34.4 billion relationships, which is more than enough for most use cases. (Neo4j could hold more than 15 nodes for each of Facebook's 2.2 billion users in a single graph.)

The Neo4j distributions provide several tools for fast lookups with Lucene, the Cypher querying language, ~~and the REST interface~~. Beyond ease of use, Neo4j is fast. Unlike join operations in relational databases or map-reduce operations in other databases, graph traversals are constant time. Like data is only a node step away, rather than joining values in bulk and filtering the desired results, which is how most of the databases we've seen operate. It doesn't matter how large the graph becomes; moving from node A to node B is always one step if they share a

relationship. Finally, the Enterprise edition provides for highly available and high read-traffic sites by way of Neo4j HA.

Neo4j's Weaknesses

Neo4j does have a few shortcomings. We found its choice of nomenclature (node rather than vertex and relationship rather than edge) to add complexity when communicating. Although HA is excellent at replication, it can only replicate a full graph to other servers. It cannot currently shard subgraphs, which still places a limit on graph size (though, to be fair, that limit measures in the tens of billions). Finally, if you are looking for a business-friendly open source license (like MIT), Neo4j may not be for you. Although the Community edition (everything we used in the first two days) is GPL, you'll probably need to purchase a license if you want to run a production environment using the Enterprise tools (which includes HA and backups).

Neo4j on CAP

The term "high availability cluster" should be enough to give away Neo4j's strategy. **Neo4j HA** is **available** and **partition tolerant** (AP). Each slave will return only what it currently has, which may be out of sync with the master node temporarily. Although you can reduce the update latency by increasing a slave's pull interval, it's still technically eventually consistent. This is why Neo4j HA is recommended for read-mostly requirements.

Parting Thoughts

Neo4j's simplicity can be off-putting if you're not used to modeling graph data.

It provides a powerful open-source API with years of production use and yet it hasn't gotten the same traction as other databases in this book. We chalk this up to lack of knowledge because graph databases mesh so naturally with how humans tend to conceptualize data. We imagine our families as trees, or our friends as graphs; most of us don't imagine personal relationships as self-referential data types. For certain classes of problems, such as social networks, Neo4j is an obvious choice. But you should give it some serious consideration for non-obvious problems as well—it just may surprise you how powerful and easy it is.
